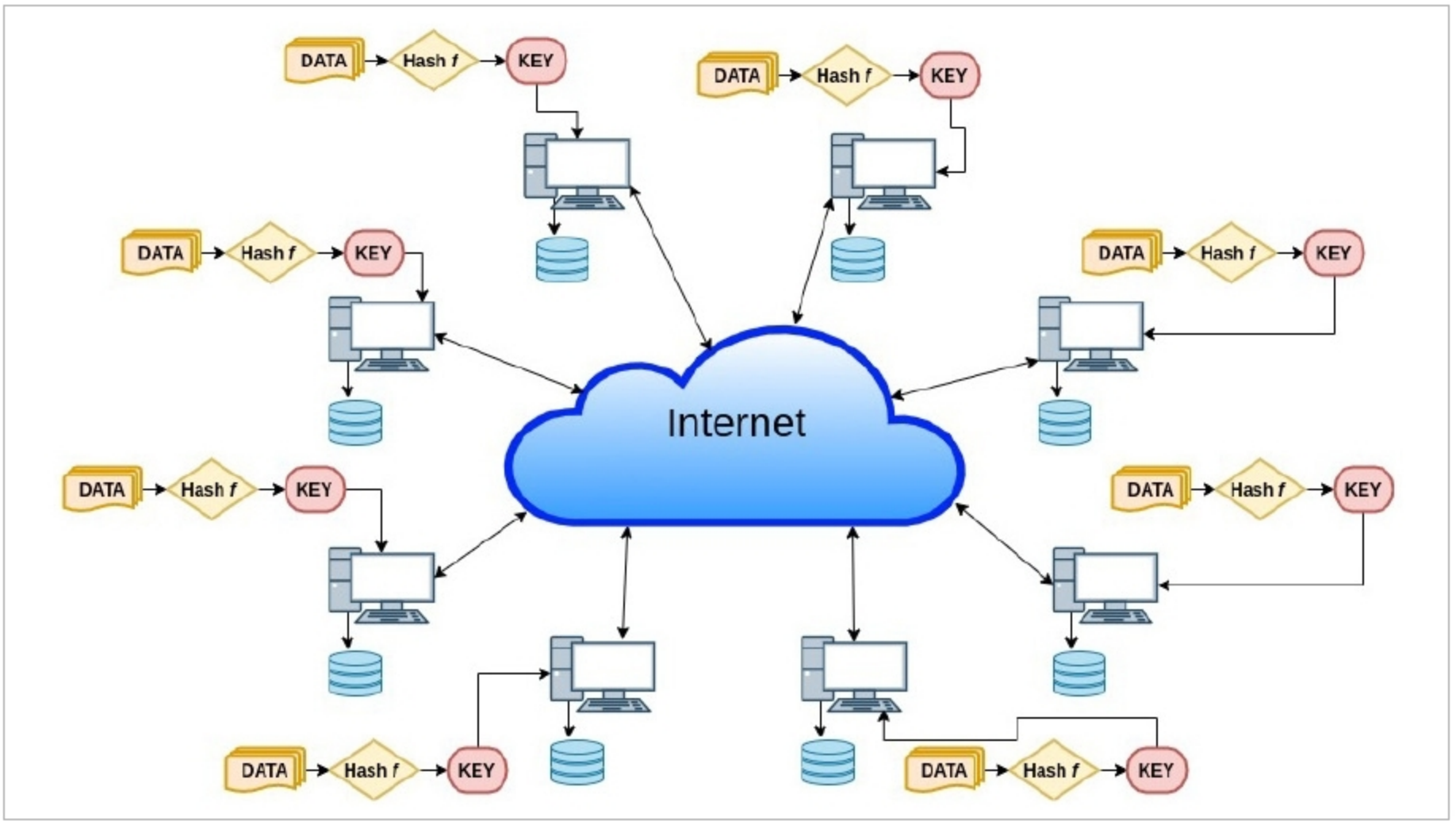

Figure 1: Decentralised routing and object location peer-to-peer wide-area application

(*https://en.wikipedia.org/wiki/CAP_theorem*) forms the foundation for the design of this application. The distributed hash tables that form a building block for P2P applications are alternatively used to make it decentralised.

Figure 1 gives an overview of the application. Here, multiple nodes share identical responsibilities with symmetric communication between them through the Internet.

Each node performs routing, storage and retrieval of values associated with keys. Each node acts as a server as well as a client having its own space for storing key-value pairs. To understand this project, prior knowledge of Design Consistent Hashing is needed *(http://www.allthingsdistributed.com/2007/10/amazons_dynamo.html)*.

Key-value store

A key-value store mainly performs two operations — *put(key,value)* and *get(key)* for storing and retrieving a value. A simpler version of key-value store (single server) can be a hash table implemented in any language to store key-value pairs. This will have limitations of performance and memory constraints.

The idea of Design Consistent Hashing is used to build a distributed key-value store where multiple servers are mapped on a ring. Given a message with a key, it's routed to a live node with a *nodeId* nearest to the key. There is still a centralisation, which is the load distributor that stores the ring and routes the message to the corresponding server, and is also involved in reorganisation of data during a node failure.

The same idea of Consistent Hashing for a distributed table forms the building block for a peer-to-peer application, which in turn can be used as a decentralised distributed hash table.

A P2P system is a distributed system having identical capabilities and responsibilities for all nodes connected to each other through the Internet. The designed application has the following features:

- A unique *nodeId* is assigned to each node.
- When supplied with a message and a key, a node performs routing of the message efficiently among all live nodes to the node with *nodeId* closest to the key.
- A message takes an expected *O(log N)* steps to reach its destination node, where *N* is the count of live nodes in the network.
- It takes the network locality into account to optimise the distance that a message travels according to a proximity (count of nearby hops a ping request travels through to reach a live node).

Design

Each node in the application's network is any computer system connected to the Internet running the application's node. Each node acts as a server as well as a client. A node interacts with other live nodes in the network and responds to client requests. Each node